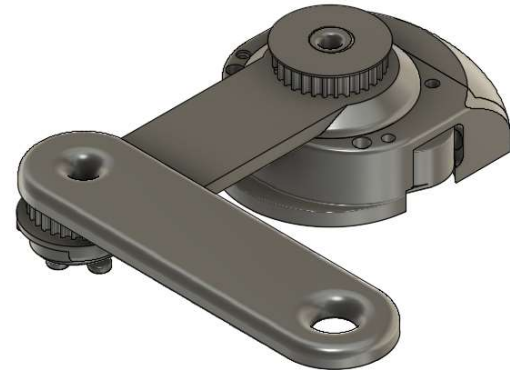




Ziel des Kapitels *Roboter-Arm*:

- Roboter-Arm mit eigenen Befehlen steuern
- Aufbau von Mechanik erleben und verstehen
- kompetitiv Aufgaben spielerisch lösen
- Netzwerk-Fähigkeiten von Python erarbeiten



Hier mit schwachen Motoren dürfen wir Fehler machen. Mit Robotik die 1 kg anheben kann, wäre zu schnell ein Loch im Tisch oder der Wand.

3.7 Robotik → Roboter-Arm

Wir hatten Robotik an den Beispielen der kleinen Elektro-Autos gesehen, konnten dort auf Eingaben reagieren, und Gegenstände in der realen Welt bewegen. Das macht Spass, aber: die Robotik geht weiter. Wir als Gesellschaft wollen hin zum *universell* einsetzbaren Roboter: Einem Roboter der von der Spülmaschine ausräumen, bis zum Bad putzen alles kann.

3.7.1 Intro (10min)

Info Roboter-Arme existieren, die uns viele alltägliche Aufgaben abnehmen könnten. Diese können nicht benutzt werden, weil die Software zu kompliziert ist. Ihr: die nächste Generation Software-Entwickler kann das hoffentlich lösen, und die Bedienung einfacher machen.

Task 39 → Ansteuern: Unser Roboter meldet sich im Informatik Netzwerk⁷ an, und hat dort eine WebPage. Gehe auf die Page `fluidncBlue.local` und bewege dort die *x* und *z* Achse, jeweils um ein paar Grad. Das kann mit den Pfeilen gemacht werden.

Wir sehen: Wenn die ganze Klasse auf ein WebFrontend zugreifen will, schmeissen wir uns gegenseitig raus. Bitte absprechen!



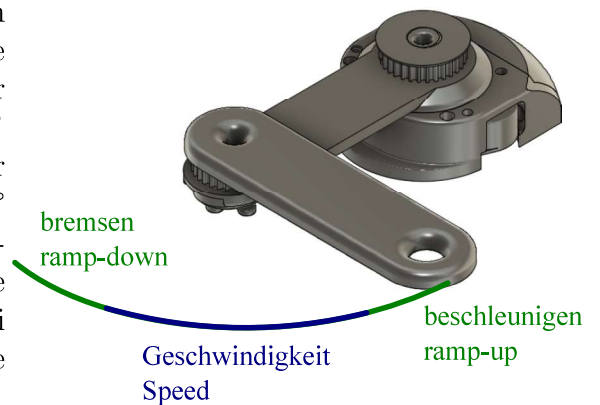
⁷WLAN mit Namen: *Informatik* und Passwort *Informatik*. Falls kein Informatik Netzwerk vorhanden ist, spannt der Roboter ein eigenes Netzwerk auf. FluidNC mit dem Passwort: 12345678

Task 40 → Lückentext: Roboter können wir z.B. per ansteuern. Dabei geben wir den einzelnen Befehle, wie sie sich bewegen sollen.

Die Bewegung der und die Bewegung ist per Mechanik verbunden. Wir müssen hier (selber) überlegen, wie sich die bewegt.

Task 41 → WebSteuerung: vom realen Roboter. Stellt den X-Wert hoch, z.B. auf 90. Oder den Z-Wert auf -60. Das können wir entweder über +10 Buttons machen, oder indem wir den Wert direkt eingeben.

Task 42 → Speed: die Geschwindigkeit kann eingestellt werden, indem wir in der Web-Oberfläche die X, Y oder Z Geschwindigkeit ändern. In der Maschinen-Steuerung hat sich “Grad pro Minute” etabliert. Ja das ist komisch, aber nun... hier können wir z.B. auf 5400 gehen, das wären 90° in einer Sekunde. Du stellst mit 5400 die Maximalgeschwindigkeit ein. Es gibt immer eine Beschleunigungs-Phase, und dann ein Teilstück bei der es mit der Geschwindigkeit von 90° pro Sekunde sich bewegt, und dann bremst es wieder ab.

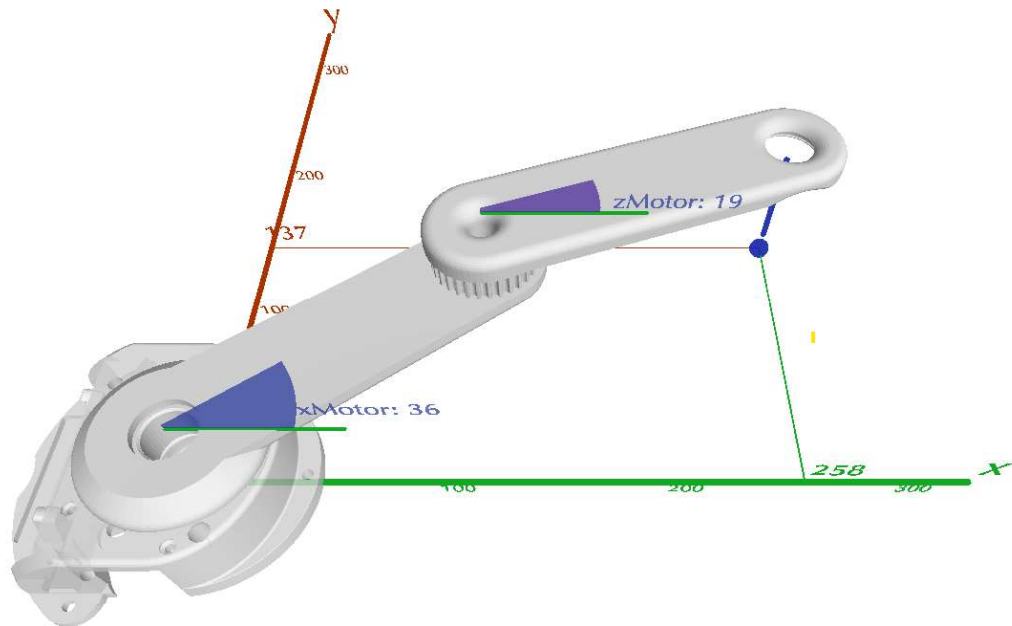


Task 43 → G-Code: Button-Klicks können nur schwierig automatisiert werden. Damit wir später Python-Befehle senden können, wollen wir alles Text-basiert los senden. Das können wir im Eingabe-Fenster machen, z.B. mit G1 x10 f3000 oder G1 X20 Z-20 F5400. Und dann dem Send-Button. Probier's grad aus.



3.7.2 Simulation (15min)

Falls zu viele Personen gleichzeitig auf den einen Roboter zugreifen, meldet sich die WebPage zum steuern nicht mehr. Dann können wir auf den Simulator sim.schooltech.ch zugreifen.



Task 44 → Simulation → steuern: in der rechten unteren Ecke hat die Simulation den **input** Button. Dort geht ein Fenster auf. Wähle hier den Winkel Z +10 um den Z-Motor 10° zu bewegen.

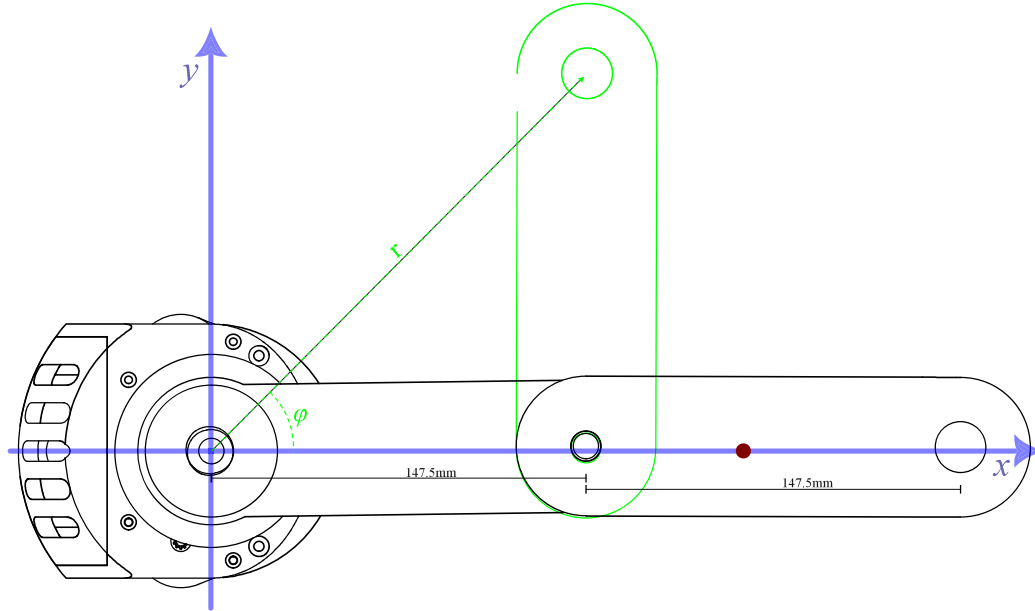
Task 45 → Simulation wechseln: Wähle beim Eingabe-Fenster und bei der simulierten Anzeige eine andere Simulation aus. Damit können wir Überschneidungen mit Kollegen vermeiden.

Task 46 → Simulation → Eingabe Üben: Erzeuge in der Simulation einen neuen Punkt (Button Dot) und fahre ihn mit dem Roboter an. Verwende dazu die verschiedenen Eingaben.

3.7.3 Bewegung der Spitze / der Hand (15min)

Wir können die Motoren (Simulation) steuern, aber uns interessiert die Hand. Hier am simulierten Roboter wollen wir das vordere Loch hin und her bewegen. Nicht mehr zufällig bewegen, sondern bewusst entscheiden wo die Hand hin fahren soll.

Task 47 → Bewegungen: Benennen wir die Bewegungen: Ganz Ausgestreckt, sei die Motor-Position 0 / 0. Also der $\mathbf{x}$ -Motor auf 0° und der $\mathbf{z}$ -Motor auf 0° . Steuer den Motor in die ‘grüne’ Position.



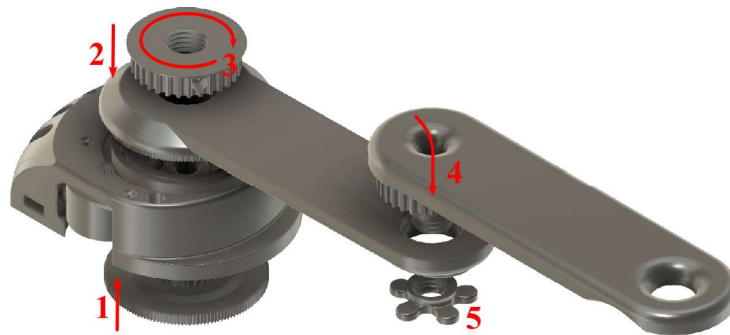
Koordinaten-System können wir so legen, dass die x Achse in die ausgestreckte Richtung geht, und die y Achse nach oben. Achtung: die Achsen x und y haben nichts mit den Motoren-Positionen $\mathbf{x}$ und $\mathbf{z}$ zu tun.

Task 48 → Winkel: Welchen Abstand r und welchen Winkel φ hat der Punkt der Hand, wenn der Unterarm um 90° angewinkelt ist? (hier grün gezeichnet).

Task 49 → Winkel auf x -Achse: welchen Motor-Winkel $\mathbf{x}$, $\mathbf{z}$ muss ich einstellen, um den roten Punkt $(x, y) = (208.6 \text{ mm}, 0.0 \text{ mm})$ zu erreichen?
b) gibt es auch andere Winkel, mit denen ich diesen Punkt erreichen kann?

Task 50 → Winkel auf x -Achse: nachdem du es berechnet hast, stelle die Winkel auf der Hardware ein, und schaue nach, ob es auf der x -Achse liegt.

Task 51 → Aufbau: (pro Tisch-Reihe einen) nehmt euch die Bauteile, die es braucht, um den Roboter aufzubauen. Schraubt ihn zusammen.

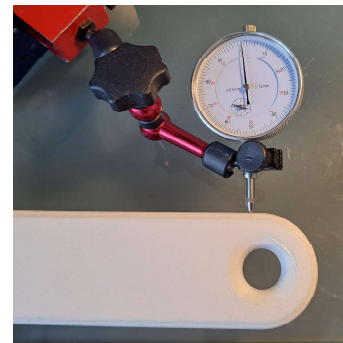


Bei mir funktioniert der Zusammenbau gemäss den folgenden Schritte relativ gut: 1: Unterarm-Zahnrad durch schieben. 2: Oberarm einklicken, so dass das Zahnrad am Motor einhakt. 3: Riemenscheibe drauf schrauben. So fest ziehen, dass die Scheibe sich nicht später unbeabsichtigt löst⁸. 4: Riemen über die Riemenscheibe legen und Unterarm einhaken, so dass der Riemen danach gespannt ist. 5: Gegen-Mutter anschrauben.

Task 52 → Ansteuern: vom selbst aufgebauten Roboter. Die Web-Adresse ist drauf geschrieben. Entweder im Informatik WLAN, oder im FluidNC WLAN.

Task 53 → Genauigkeit: überprüfe mit einer Mess-Uhr, wie genau der Roboter ist. Erstens die Wiederholbarkeit einer angesteuerten Position, zweitens die Auflösung der Positionen. Denn die Motoren haben eine gewisse Ungenauigkeit, die Zahnräder sind nicht ganz exakt. Trotzdem sollte der Roboter auf 0.05 mm genau sein.

Bei mir funktioniert es nur, wenn ich den Roboter mit einer Zwingen am Tisch fixiere.



Task 54 → Torwand: Lege einen Ball auf $x = 12\text{ cm}$ und $y = 0\text{ cm}$. Versuche, den Ball möglichst gerade entlang der x Achse zu 'kicken'.

3.7.4 G-Code

Mit Maus-Klicks können wir je einen Motor bewegen. Wenn wir gleichzeitig mehrere Motoren bewegen wollen, brauchen wir Befehle. Die Web-Oberfläche FluidNC hat dafür sogar ein Eingabe-Fenster:

⁸Die Rotations-Bewegung würde das Gewinde öffnen. Deshalb muss eventuell eine Sicherungs-Schraube verwendet werden. Oder fest genug anziehen.

1 `G1 x10 z20 f3000`

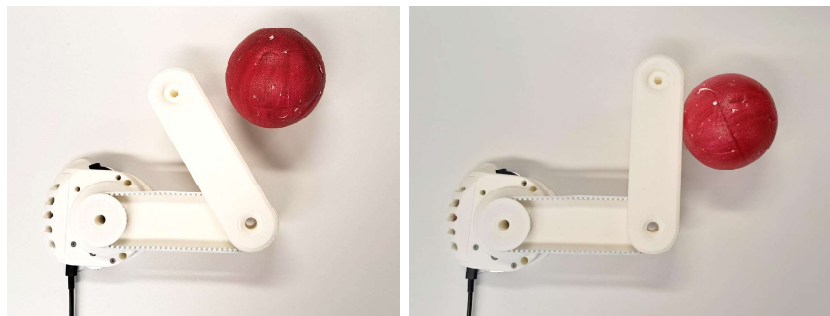
Der Befehl `G1 Gehe Zu`, dann die `x` und `z` Motor-Winkel, und dann die Feed-Rate $3000 \frac{\circ}{\text{min}}$



Damit können wir kombinierte `x` und `z` Bewegungen laufen lassen.

Task 55 → Bewegung: Fahre zur Position $(0, 0)$ und dann zur Position $90, z = 180$. Eingabe-Vereinfachung: mit der Pfeiltaste $\uparrow$ kann der letzte eingegebene Befehl wiederhergestellt werden.

Task 56 → Torwand: führe den Kick entlang der x Achse mit `G1` Befehlen aus. Variiere die Geschwindigkeit.



Task 57 → Geschwindigkeit: kann mit `f7000` natürlich erhöht werden. Der Roboter hat einen Maximal-Wert. Es kann sein, dass dieser Maximal-Wert dann die Feed-Rate nicht akzeptiert.

Wenn die Motoren nicht mehr so schnell können, fangen sie an zu stottern, und die Bewegung wird unzuverlässig. Eventuell bewegt sich noch der eine Motor, und der andere bleibt hängen. Teste es aus, mit der Bewegung von $0, 0$ zu $x = 90, z = 190$ `f19000`.

Info Die Geschwindigkeiten hängen von verschiedenen Einstellungen und Reibungen des Getriebes ab. Die Spannung des Netztes spielt auch eine Rolle. Der Roboter 'verlangt' 12 V vom Netzteil. Aber einige Netzteile liefern nur 5 V. Dann hat der Motor weit weniger Leistung und bleibt schneller hängen.

Task 58 → Genauigkeit: Wie viele μm bewegt sich das Handgelenk, wenn der z-Motor um 0.1° gedreht wird? Jetzt kannst du genaue Werte eingeben, dann können wir auch 0.04° testen. Eventuell bleiben so kleine Werte halt im Wackeln des Motors versteckt.

Task 59 → Traglast: wir haben bisher die Bewegung ohne ‘Hand’ und ohne Greifer durchgeführt. Lege ein oder zwei Stifte auf den ausgestreckten Arm, um die Stabilität zu testen. Wie viel Gramm kann der Arm tragen, ohne zu kippen?

Überlege dir (mithilfe der Hebelgesetze) wie schwer ein Roboter sein müsste, der am ausgestreckten 30 cm Arm 1 kg tragen sollte.

3.7.5 Python Code Sender

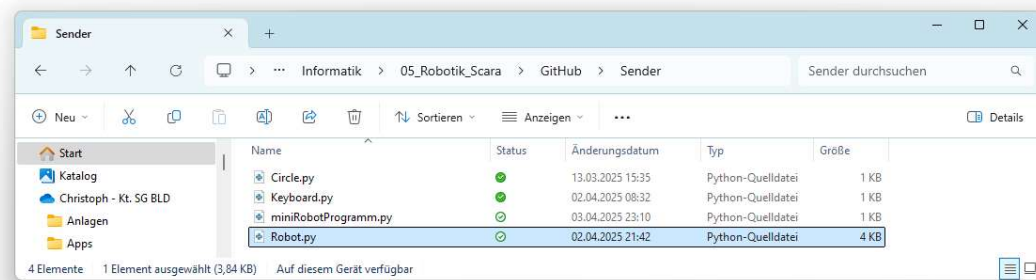
Wir haben G-Code, und wollen das jetzt von unserem Laptop aus an den Roboter senden. Da ist es praktisch, dass Python alle Techniken beherrscht: Wir können normal auf's WLAN zugreifen.

Genauso wie wir die Informatik mit einem `Turtle` Programm gestartet haben, wollen wir alles, was zum Roboter gehört, in einer `from Robot import *` Klasse verpacken. Wir wollen uns nicht in die Kommunikation kümmern. Aber: Den Roboter gibt es noch nicht allgemein in Python, hier müssen wir unsere `Robot.py` Datei selber installieren.

Task 60 → Robot.py kopieren: Um mit dem Roboter in Python genauso zu arbeiten wie mit Python, kopieren wir uns die `Robot.py` von

https://github.com/ChKendel/Robot_2Motor_Scara/tree/main/Sender

in unser Arbeits-Verzeichnis.



Task 61 → Python Sender: Wir stellen am Roboter `fluidnc.local` den Motor auf `x=20` und `z=-35` ein. . Verwende die eigenen Klasse `Robot.py` im Arbeits-Verzeichnis mit dem hier angegebenen Beispiel code.

```
1 from Robot import *
2
3 r = Robot("fluidnc.local", 80)
4
5 r.gotoMotorXZ(20, -35)
```

Beachte: Wir verwenden Netzwerk-Zugriffe, gestartet mit `Robot`.

Task 62 → Python Sender: Sende dann die Motor-Positionen `x40` und `z40`.

Info: Klassen Hier rufen wir die Klasse `Robot` auf. Das ist technisch nichts anderes, als damals der Aufruf von `Turtle`. Alle Funktionen können

dann über den Speicherbereich `robot` erreicht werden.

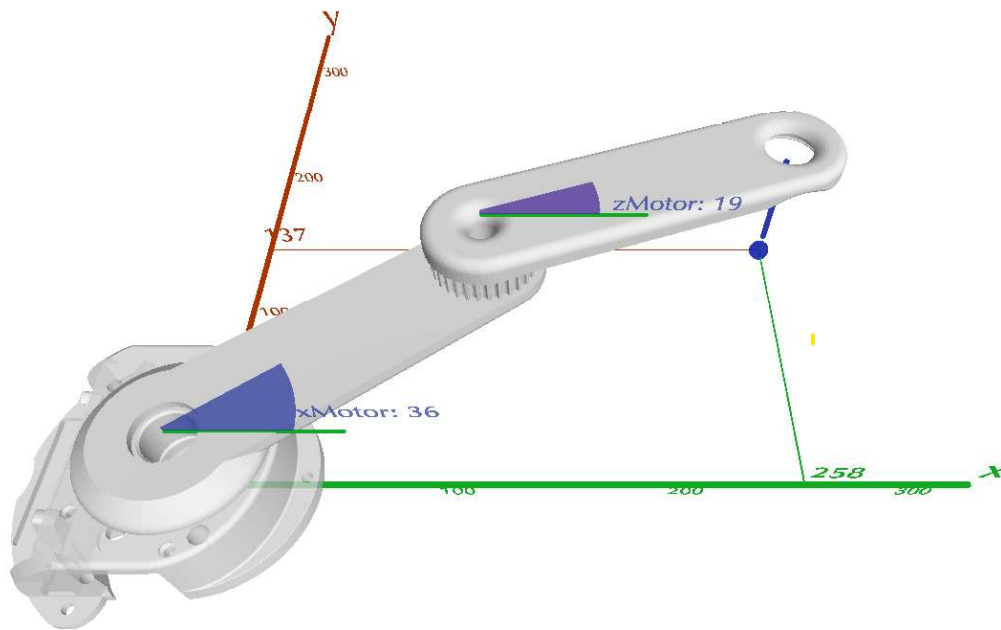
```
1 from Robot import *
```

Dazu muss natürlich die Datei `Robot.py` für das Python sichtbar sein. Da es keine Normale Python-Erweiterung ist, müssen wir manuell die `Robot.py` ins Arbeitsverzeichnis kopieren: dort hin, wo unsere Programm-Datei rum liegt.

Task 63 → Warten: Schicke zwei Befehle los. Erst auf $x = 30, z = 40$ dann $x = 0, z = 0$. Lass das Programm dazwischen 1s warten. Schau' im Netz, wie du `time` verwenden kannst, um Python warten zu lassen.

Task 64 → Code Lesen: Was steht in der Klasse `Robot.py` drinnen? Erkläre zwei der Funktionen (hier schriftlich) in eigenen Worten, so dass Kollegen den Text verstehen.

Task 65 → Anderen Roboter: Verwende statt den `fluidnc.local` den simulierten Roboter `r = Robot("sim.schooltech.ch", 80, robotNr = 4)`. Dieser simulierte Roboter kann auf <https://sim.schooltech.ch> angeschaut werden. Beachte, dass hier noch die `RoboterNr` eingestellt werden muss.



Task 66 → Hin–Her: schreibe ein Python Programm, das auf G1 x36 z19 f4000 geht, und dann zu G1 x0 z0 f3500 zurück läuft.

Task 67 → Simulation: Teste das Python Programm, indem du auf den simulierten Roboter zugreifst. Hier werden nicht nur die Motor–Koordinaten, sondern auch Karthesische Koordniaten gut sichtbar.

Task 68→ Tastatur–Eingabe: Der Verbindungs–Aufbau zwischen Python und dem Web Frontend geht ca. 2s. Das ist zu lange als das man damit eine präzise Tastatur–Steuerung bauen könnte. Deshalb die Idee: Einmal die Verbindung aufbauen, und dann auf eine Tastatur–Eingabe warten:

```

11     while True:
12         # Wait for key press
13         if keyboard.is_pressed('a'):
14             r.CoordX += 1
15             r.gotoCoordXY()
16             time.sleep(0.006)

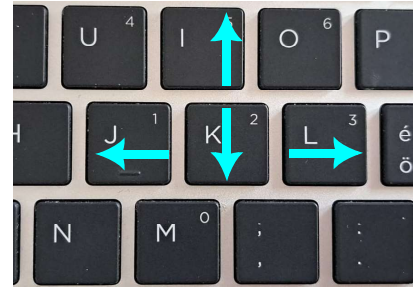
```

Eventuell muss dazu noch `pip install keyboard` installiert werden.

Task 69 → Tastatur–Steuerung: mit a,s für grösseren und kleineren x Motor–Winkel, sowie q,w für grösseren und kleineren z Motor–Winkel.

Task 70 → Tastatur-Steuerung JKL: die Buchstaben JKL und I können als Cursor verwendet werden. Was wäre nötig, um damit Links-Rechts-Bewegungen zu machen?

Ich bevorzuge einen Cursor auf der Tastatur (statt im Pfeil-Block) denn so kann ich mir weitere Funktionen oder ein Macro auf u oder o legen.



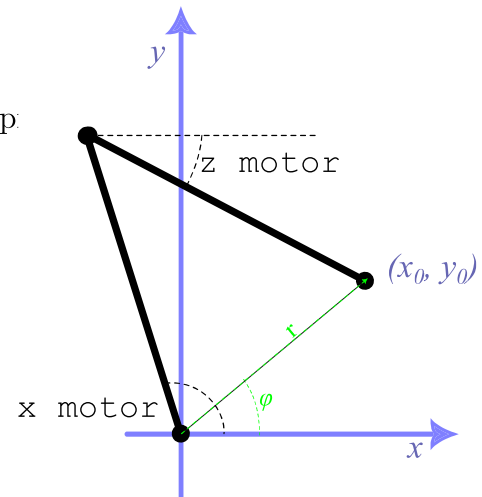
Task 71 → Eingabeformen: Welche Steuerungen würdet ihr bevorzugen? Stift-Eingabe? Joystick? Handy-mit-Lagesensor? Vorschläge:

-
-

Ihr kennt Python. Wie können wir diese Eingabeform p

Task 72 → Koordinaten: Auf Seite 83 hatten wir gesehen, dass wir kartesische Koordinaten verwenden können. Klar, (x_0, y_0) Koordinaten sind viel einfacher einzugeben, also Motor-Winkel x und z . Wie die zusammen hängen? Relativ einfach über Winkel-Funktionen $\sin(..)$, $\cos(...)$ und so. Das behandelt jeder Kantischüler in der Mathematik.

Finde hier in der Skizze die rechtwinkligen Dreiecke, mit denen wir die Motor-Winkel x und z bestimmen können.



Task 73 → Robot – Quellcode: lese den Python Code der `Robot.py` und schaue gemeinsam mit dem Sitznachbarn die `gotoCoordXY` Funktion an. Beschreibe in eigenen Worten:

Der Ellenbogen-Winkel wird ausgerechnet indem ...

Der MotorWinkel x wird ausgerechnet indem ...

Info Mit der `Robot.py` haben wir die Möglichkeiten, im kartesischen x, y -Koordinaten-System zu sein, und den Roboter an die entsprechende Position hin schicken zu können.

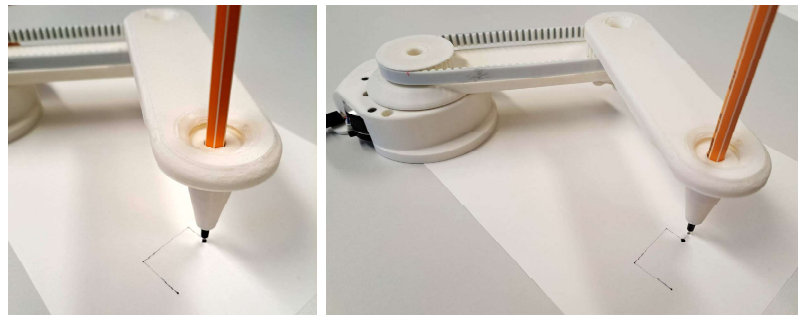
Task 74 $\rightarrow$ Verwende `Robot.py`: um ein Viereck in die Luft zeichnen zu lassen: $(100, 0) \rightarrow (120, 0) \rightarrow (120, 40) \rightarrow (100, 40) \rightarrow (100, 0)$.

Task 75 $\rightarrow$ Steuerung: Schreibe ein Programm, das mit i,k die ‘Hand’ in y Richtung bewegt, und mit j,l in x Richtung. Damit können wir mit der Tastatur den Roboter ziemlich genau steuern.

Task 76 $\rightarrow$ Kicker: Schreibe ein Programm, das zwei Bälle von zwei verschiedenen Positionen gerade (entlang der x Richtung) los kickt.

Ausblick dieser Roboter soll uns trainieren, mit Winkeln und Hebel-Kräften umgehen zu können. Natürlich braucht es wesentlich mehr Traglast (1 kg oder so) und mehr Reichweite (der menschliche Arm kommt ca. 50 cm weit). Solche Robotik gibt es in der Industrie. Solche Robotik wird auch im Haushalt kommen. Aber: Die Gefahren von falscher Handhabung sind zu gross. Es gibt zu schnell Löcher in Wänden oder zerschlagene Tische. Hier muss sich die Bedienung wesentlich verbessern.

Task 77 $\rightarrow$ Stift Plotter: Steuer die Bewegungen, um mit einem Stift (an der Roboter-Spitze) eine Figur zu zeichnen. Verwende die Bewegung mit den verschiedenen Koordinaten-Systemen von Task 72 um gerade Striche zu zeichnen.



Task 78 $\rightarrow$ Anwendungen: Mit dem neuen Wissen zu mechanischen Bewegungen: Welche Anwendungen, welche Situationen, welche Szenarien siehst du, in denen Robotik (bei passender Bedienerfreundlichkeit) sinnvoll wären?

-
-

FluidNC Die Hardware des Roboters basiert auf einem ESP32. Dieser steuert über TMC 2209 Motor-Treiber den Stromfluss zu den Nema14 Stepper Motoren. Auf dem Controller ist der die FluidNC Software installiert. FluidNC hat eine grosse Community, die das weiter entwickelt.

Wenn euer Python Programm irgend einen GCode produzieren kann, dann kann der von vielen anderen Robotern übernommen werden. Wenn Deine Idee eine Benutzer-Eingabe erkennt und daraus GCode erzeugt, kann das in Zukunft verwendet werden.